N045 AI 기능을 서비스로 설계하기_학습서

Chapter01

AI 기능을 서비스로 설계하기

코딩 에이전트와 함께라면 AI 기능 하나를 만드는 데 하루가 걸리지 않습니다. 그런데 만든 기능을 동료나 고객이 실제로 계속 쓰게 하는 일은 훨씬 오래 걸립니다. 주소를 공유해도 며칠 뒤에는 접속하는 사람이 없고, 사내에서 쓰려고 하면 "회사 문서를 외부 LLM에 보내도 되나요?"라는 질문에서 막히고, 외부에 공개했더니 모르는 사람이 내 API 키 비용으로 기능을 호출하기도 합니다. 몇 주 뒤 기능을 고치려고 하면 어느 파일을 고쳐야 할지 모르는 일도 생깁니다. 앞의 문제들은 제공 방식에서, 마지막 문제는 아키텍처에서 생깁니다.

오전에는 AI 기능 하나를 정해 사용 맥락을 분석하고, 제공 채널과 처리 방식, AI 오류 대응을 정한 뒤, 코드 없이 프로토타입으로 사용성 테스트를 합니다. 오후에는 웹앱으로 제공할 때 필요한 아키텍처 다이어그램, 시퀀스 다이어그램, 데이터 모델을 읽는 방법을 배우고, 오전에 정한 내 서비스의 아키텍처를 직접 설계합니다. 마지막에는 오전과 오후의 결정을 PRD(제품 요구사항 문서) 한 장으로 묶습니다. 배포나 설치는 없습니다.

주관식 퀴즈

Q. [시작 질문] 자주 쓰는 AI 서비스나 직접 만들어 본 AI 기능 하나를 떠올려 보세요. 사용자는 그 기능을 어떤 경로로 쓰고 있는지, 그리고 버튼 하나를 누르면 내부에서 어떤 호출이 어떤 순서로 일어날지 아는 만큼, 짐작하는 만큼 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. 사용 맥락을 분석하고, 기성 AI 도구부터 웹앱까지 다섯 가지 제공 채널 가운데 맞는 것을 고를 수 있습니다.
2. LLM 처리 시간에 맞춰 동기 처리, 비동기 처리, 스케줄 실행 가운데 처리 방식을 고르고, AI 오류 대응 장치를 정할 수 있습니다.
3. 오즈의 마법사 테스트와 기성 AI 도구 프로토타입으로 코드 없이 사용성 테스트를 할 수 있습니다.

4. 아키텍처 다이어그램, 시퀀스 다이어그램, 데이터 모델로 웹앱의 구조를 읽고, 내 서비스의 아키텍처를 설계할 수 있습니다.
5. 접근 제어, 데이터 처리, 비용, 파일럿 계획, 아키텍처를 담은 PRD 한 장을 쓸 수 있습니다.

Chapter02

사용 맥락 분석

AI 기능을 만들고 나면 "주소를 보내 줄 테니 써 보세요"라고 말하기 쉽습니다. 그런데 사용자는 하루 대부분을 이미 쓰는 도구 안에서 보냅니다. 회사라면 업무 메신저, 메일, 문서 도구이고, 개인이라면 휴대폰 메신저, 노트 앱, 몇 개의 서비스입니다. 별도 주소로 접속해야 하는 기능은 초기에는 쓰이다가 사용률이 빠르게 떨어집니다. 그래서 제공 채널을 정하기 전에 사용 맥락(context of use), 곧 사용자가 언제 어떤 기기와 도구를 쓰고 있을 때 이 기능이 필요한지를 먼저 분석합니다.

항목	강의 복습 노트와 퀴즈의 예
대상 사용자	이 과정의 수강생. 하루 7시간 강의를 듣고 노트 앱에 필기한다
필요한 시점	강의 직후(정리), 다음 날 아침 강의 전(복습)
사용 중인 기기와 도구	강의 직후에는 노트북의 노트 앱, 다음 날 아침에는 휴대폰
결과를 받은 뒤의 다음 작업	복습 노트를 노트 앱에 저장하고, 다음 날 퀴즈를 푼다
현재 방식	필기를 다시 읽으며 30분쯤 걸려 정리하고, 대부분 미룬다

표를 채우면 제공 채널에 대한 요구사항이 보입니다. 복습 노트를 만드는 시점은 노트북 앞이지만 퀴즈를 푸는 시점은 다음 날 휴대폰이므로, 결과가 휴대폰에서 다시 열 수 있는 곳에 남아야 합니다. 노트북에서만 열리는 화면이나, 대화가 금방 밀려 올라가는 단체 대화방은 이 요구사항에 맞지 않습니다. 사용자 무리가 여럿이면(필기한 사람, 필기를 공유받는 스터디원) 기능을 직접 호출하는 무리를 먼저 분석하고, 결과를 받아 보기만 하는 무리는 "다음 작업" 칸에서 함께 다룹니다.

실습: 사용 맥락

오늘 하루 동안 설계할 AI 기능을 정합니다. 기본은 내가 만들고 싶은 서비스입니다. 떠오르지 않으면 위 표의 예시인 "강의 필기를 넣으면 복습 노트와 퀴즈를 만들어 주는 기능"으로 진행합니다. 이 기능은 같은 반 수강생 모두가 실제 사용자라서, 옆 사람에게 인터뷰하고 사용성 테스트를 하면 짐작이 아닌 실제 사용 데이터를 얻을 수 있습니다.

대상 사용자 무리 하나를 정하고 표의 다섯 항목을 채웁니다. 실제 사용자에게 확인하지 못한 칸은 "가정"이라고 표시해 두고, 아래 인터뷰와 데스크 리서치로 가정을 하나씩 줄입니다.

직접 확인할 곳: 사용자 인터뷰

1. 대상 사용자에 가장 가까운 사람 한 명에게 10분을 요청합니다. 강의 필기 기능이라면 옆 수강생이 바로 실제 사용자입니다. 내 서비스의 대상이 수강생과 다르다면 수업 중에는 그 대상에 가까운 수강생과 하고, 수업 뒤에 가족이나 지인 가운데 실제 대상에게 같은 질문을 메신저로 보냅니다.
2. 기능을 설명하지 않고, 이 작업을 마지막으로 한 순간을 묻습니다. 예: "어제 강의 끝나고 필기는 어떻게 정리했어요? 다시 본 적 있어요?"
3. 그때의 장소, 사용한 기기와 도구, 결과를 어디에 두었는지, 걸린 시간을 차례로 묻고 표에 옮겨 적습니다.
4. "이런 기능이 있으면 쓰시겠어요?"는 묻지 않습니다. 대부분 긍정으로 답해서 판단 근거가 되지 않습니다.

데스크 리서치

실제 사용자를 충분히 만날 수 없을 때는 이미 공개된 자료에서 사용자의 행동과 불편을 찾습니다. 현업에서도 사용자 인터뷰 전에 먼저 하는 조사이고, 데스크 리서치(desk research)라고 부릅니다. 사용자가 직접 남긴 글이라서, 내가 상상한 불편보다 구체적인 표현과 상황이 나옵니다.

직접 확인할 곳: 데스크 리서치 진행

1. 비슷한 문제를 푸는 서비스를 두세 개 찾습니다. 강의 필기 기능이라면 노트 앱, 플래시카드 앱, AI 요약 서비스입니다.
2. 앱 스토어나 서비스 후기에서 별점 2-3점짜리 리뷰를 20개쯤 읽습니다. 별점 1점은 결제나 오류 불만이 많고, 2-3점에 "이건 좋은데 이게 불편하다"는 구체적인 사용 맥락이 많습니다.
3. 커뮤니티의 질문 글을 검색합니다. 예: "강의 필기 정리 어떻게 하세요", "복습 퀴즈 만드는 법". 질문자가 지금 어떤 도구로 어떻게 하고 있는지를 적습니다.
4. 찾은 내용은 출처 링크와 함께 사용 맥락 표의 해당 칸에 옮기고, "가정" 표시를 "리서치"로 바꿉니다. 한 사람의 글만으로는 바꾸지 않고, 비슷한 내용이 두 곳 이상에서 나올 때 바꿉니다.

코딩 에이전트에게 맡길 때: 인터뷰 질문 만들기

```
<기능을 한 문장으로>를 <대상 사용자>가 쓴다고 하자.
이 사용자가 이 작업을 할 때의 기기와 도구, 결과를 받은 뒤의 다음 작업, 현재 방식과 걸리는 시간을 확인할 인터뷰 질문을 다섯 개 만들어 줘.
"이 기능을 쓰겠느냐"처럼 의견을 묻는 질문은 빼고, 최근에 실제로 한 행동을 묻는 질문으로 만들어 줘.
답은 지어내지 마.
```

다섯 가지 제공 채널

같은 AI 기능을 사용자에게 제공하는 채널은 여러 가지이고, 채널마다 개발 기간과 직접 제어할 수 있는 범위가 다릅니다.

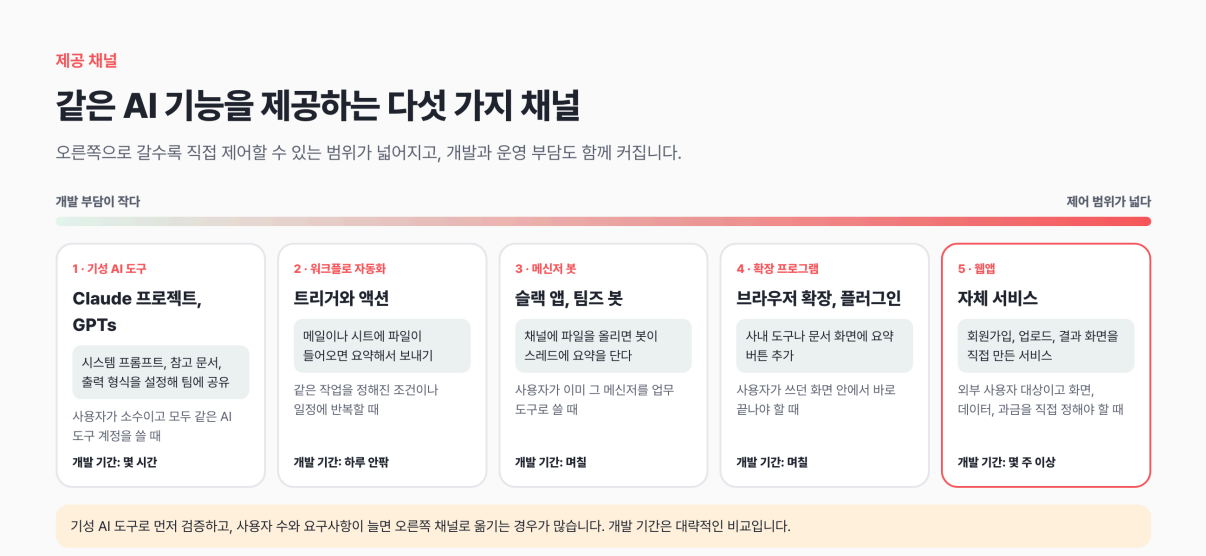


그림: AI 기능의 다섯 가지 제공 채널. 오른쪽으로 갈수록 제어 범위와 개발 부담이 함께 커진다

기성 AI 도구는 Claude 프로젝트나 GPTs처럼 이미 있는 AI 서비스 안에 시스템 프롬프트, 참고 문서, 출력 형식을 설정해 공유하는 방식입니다. 코드가 없으니 몇 시간이면 배포할 수 있지만, 사용자가 모두 그 도구의 계정을 가지고 있어야 하고 화면, 데이터 저장, 접근 범위는 그 도구가 정한 대로 따릅니다. 웹앱은 화면, 인증, 데이터, 과금을 모두 직접 정할 수 있는 대신 개발, 배포, 운영을 모두 맡아야 합니다. 이 과정의 뒤에서 배우는 배포 파이프라인, 비용 한도, 모니터링은 웹앱을 골랐을 때 필요한 작업입니다.

실무에서는 기성 AI 도구로 먼저 검증하고, 사용자가 늘거나 요구사항이 생길 때 오른쪽 채널로 옮기는 경우가 많습니다. 전환 조건은 대개 셋 가운데 하나입니다. 도구 계정이 없는 사용자도 써야 할 때, 사용량과 비용을 직접 관리해야 할 때, 결과를 메신저 채널이나 사내 시스템에 자동으로 연동해야 할 때입니다. 처음부터 웹앱을 만들면 사용자가 정말 이 기능을 쓰는지 확인하기 전에 몇 주를 쓰게 됩니다.

같은 기능, 다른 요구사항

제공 채널은 기능이 아니라 사용자가 정합니다. 아래 표는 강의 복습 노트와 퀴즈라는 같은 기능을 혼자 복습하는 수강생, 5명짜리 스터디, 온라인 강의를 듣는 외부 학습자에게 제공할 때 달라지는 요구사항을 비교한 것입니다.



같은 기능, 다른 사용자

강의 복습 노트를 누구에게 제공하느냐에 따라 달라지는 것

LLM이 하는 일은 셋 다 "필기나 강의 내용으로 복습 노트와 퀴즈를 만든다"로 같습니다. 달라지는 것은 그 바깥의 요구사항입니다.

	개인 혼자 복습하는 수강생	스터디 같은 과정을 듣는 5명, 단체 대화방이 있다	외부 학습자 온라인 강의를 듣는 누구나
제공 채널	기성 AI 도구. 프로젝트에 지침과 출력 형식 설정	메신저 봇. 단체 대화방에 퀴즈를 올린다	웹앱. 회원가입 후 필기나 강의 녹화를 업로드
처리 방식	동기. 필기를 넣으면 바로 받는다	스케줄 실행. 매일 밤 9시에 그날 퀴즈를 올린다	비동기. 2시간 녹화를 처리하고 끝나면 알림
AI 오류 대응	출처 표시. 직접 읽고 수정	피드백. 틀린 문제에 👎, 봇 운영자가 확인	출처 표시(녹화 시각)와 편집 화면, 신고
접근 제어	본인 계정	단체 대화방 멤버만	회원 인증. 자기 자료만 조회
데이터 처리	본인 계정의 대화 기록에만 남는다	각자의 필기가 LLM 제공사로 전송된다. 스터디원 동의	타인의 강의 자료 저장. 보존 기간, 삭제 기능, 저작권 고지
비용	개인 구독료 안에서	운영자 한 명의 API 키. 월 한도와 분담 방법	사용자 부담. 요금제와 무료 쿼터
성공 지표	정리 시간 30분에서 5분	퀴즈 참여율 주 5일 중 4일	주간 재사용률

그림: 강의 복습 노트와 퀴즈를 개인, 스터디, 외부 학습자에게 제공할 때 달라지는 요구사항

LLM이 하는 일은 셋 다 "필기나 강의 내용으로 복습 노트와 퀴즈를 만든다"로 같지만 표의 나머지 행은 모두 다릅니다. 혼자 복습하는 수강생이라면 개발할 것이 없고, 기성 AI 도구 프로젝트 하나로 충분합니다. 스터디는 모두 같은 단체 대화방에 있으니 메신저 봇이 맞고, 매일 정해진 시각에 그날 퀴즈를 올리는 스케줄 실행이 "함께 푼다"는 목적에 맞습니다. 대신 각자의 필기가 LLM 제공사로 전송된다는 것을 스터디원에게 알리고 동의를 받아야 하고, 봇 운영자 한 명의 API 키로 비용이 나가므로 월 한도와 분담 방법을 정해야 합니다. 외부 학습자에게 공개하면 타인이 올린 강의 녹화와 자료를 저장하는 서비스가 되므로 회원 인증, 보존 기간, 저작권 고지, 과금이 모두 필요하고, 그래서 웹앱이 됩니다. 2시간짜리 녹화를 처리해야 하니 처리 방식도 비동기가 됩니다. 표의 일곱 행은 오늘 PRD에서 하나씩 채울 항목이기도 합니다.

실습: 제공 채널

앞에서 정한 사용자와 기능으로 제공 채널 후보를 두 개 고릅니다. 하나는 가장 빨리 사용자에게 써 보게 할 수 있는 채널, 다른 하나는 사용자가 늘었을 때 옮겨 갈 채널입니다.

기준	후보 1	후보 2
사용자가 이미 쓰는 도구 안에서 끝나는가		
사용자에게 새로 필요한 것 (계정, 설치)		
직접 제어할 수 없는 것 (화면, 데이터 저장, 접근 범위)		
첫 사용까지 걸리는 시간		

사용자가 이미 가진 계정(쓰고 있는 AI 도구, 메신저, 노트 앱)도 함께 적습니다. 사용자에게 새 유료 계정이나 설치가 필요한 채널은 첫 버전으로 맞지 않는 경우가 많습니다.

객관식 퀴즈

Q. 스터디원 5명이 "강의 자료를 질문하면 해당 슬라이드 번호와 함께 답해 주는 기능"을 원합니다. 5명 모두 같은 AI 도구의 유료 계정이 있고, 강의 자료는 PDF 20개입니다. 가장 먼저 시도할 제공 채널로 알맞은 것은 무엇인가요?

• 1. 회원가입과 관리자 화면이 있는 웹앱을 개발한다
• 2. 기성 AI 도구의 프로젝트에 강의 자료와 응답 규칙(슬라이드 번호 필수)을 설정해 5명에게 공유한다
• 3. 강의 플랫폼 화면에 브라우저 확장 프로그램으로 질문 버튼을 추가한다
• 4. 모든 기수의 수강생이 쓸 메신저 봇을 개발한다

- •

  1. 회원가입과 관리자 화면이 있는 웹앱을 개발한다

- •

  2. 기성 AI 도구의 프로젝트에 강의 자료와 응답 규칙(슬라이드 번호 필수)을 설정해 5명에게 공유한다

- •

  3. 강의 플랫폼 화면에 브라우저 확장 프로그램으로 질문 버튼을 추가한다

- •

  4. 모든 기수의 수강생이 쓸 메신저 봇을 개발한다

Chapter03

동기 처리, 비동기 처리, 스케줄 실행

일반 웹 서비스는 요청에 1초 안에 응답합니다. LLM 호출은 짧은 응답도 몇 초가 걸리고, 긴 녹음을 요약하거나 문서 여러 개를 분석하면 몇 분이 걸립니다. 그래서 처리 시간과 사용자가 결과를 필요로 하는 시점에 맞춰 처리 방식을 정합니다.



그림: 동기 처리, 비동기 처리, 스케줄 실행

강의 한 번 분량의 필기 요약은 동기 처리로 충분하지만, 2시간짜리 강의 녹화를 요약하는 기능을 동기로 처리하면 사용자는 몇 분 동안 화면을 떠나지 못하고, 그 사이 연결이 끊기면 결과를 잃습니다. 이런 기능은 요청을 작업 큐에 넣고 바로 접수 응답을 준 뒤 워커가 처리하는 비동기 방식이 맞고, 처리 상태 표시와 완료 알림을 받을 채널(메신저, 메일)을 함께 정해야 합니다. 매주 같은 형식으로 필요한 주간 보고 초안은 사용자 요청 없이 정해진 시각에 실행하는 스케줄 실행이 더 편합니다.

처리 방식과 함께 출력 형식도 정합니다. LLM에게 "요약해 줘"라고만 하면 매번 길이와 순서가 다른 결과가 나오고, 사용자는 받을 때마다 필요한 부분을 찾아 다시 정리해야 합니다. 출력 형식은 사용자가 결과를 받은 뒤의 다음 작업에 맞춰 항목, 순서, 길이, 값이 없을 때의 표기로 정합니다. 다음 날 휴대폰으로 퀴즈를 푸는 것이 다음 작업이라면, 퀴즈는 휴대폰 화면에서 읽을 수 있는 길이여야 하고 정답은 문제와 분리되어 있어야 합니다. 이렇게 정한 형식은 시스템 프롬프트에도, 웹앱을 만들 때의 응답 스키마에도 그대로 들어갑니다.

AI 오류 대응

LLM은 그럴듯한 오류를 냅니다. 필기에 없는 설명을 개념 정의에 덧붙이거나, 퀴즈의 정답을 틀리게 적는 식입니다. 이런 오류는 없앨 수 없으므로, 오류가 사용자에게 그대로 전달되지 않게 제공 방식 안에 대응 장치를 둡니다.

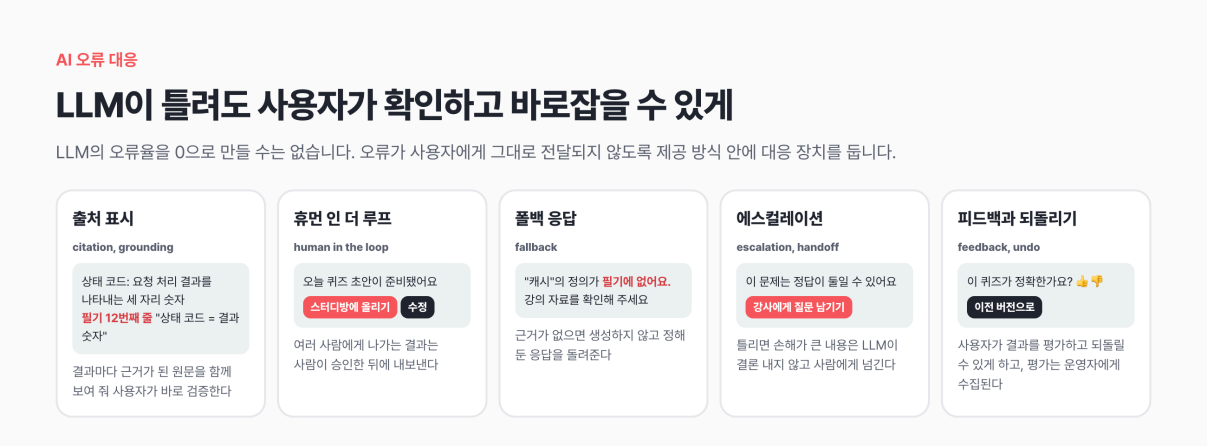


그림: AI 오류 대응 장치 다섯 가지: 출처 표시, 휴먼 인 더 루프, 폴백 응답, 에스컬레이션, 피드백과 되돌리기

다섯 가지를 모두 넣을 필요는 없고, 오류의 영향도가 클수록 많이 넣습니다. 혼자 보는 복습 노트라면 출처 표시 하나로 충분하지만, 스터디 대화방에 올라가는 퀴즈는 휴먼 인 더 루프로 봇 운영자가 확인한 뒤 올리고, 정답이 확실하지 않은 문제처럼 틀리면 여러 사람이 잘못 외우게 되는 내용은 에스컬레이션까지 넣습니다. 장치가 늘수록 사용자가 확인할 단계도 늘어나므로 편의성과 함께 따집니다. 피드백 장치로 모은 평가는 나중에 LLM 응답 품질을 평가하는 데이터로 다시 쓰입니다.

실습

내 기능의 처리 방식과 출력 형식을 정하고, 결과가 나오기까지 사용자가 보는 화면이나 메시지를 순서대로 적습니다. 그다음 발생할 수 있는 오류를 세 가지 적고, 영향도(낮음, 중간, 높음)와 대응 장치를 정합니다.

오류 유형	영향도	대응 장치	사용자에게 보이는 문구
예: 필기에 없는 설명을 개념 정의에 덧붙임	중간	출처 표시, 폴백 응답	"정의: … (필기 12번째 줄)"

문구 칸은 실제로 사용자에게 나갈 문장으로 씁니다. "오류가 발생했습니다"보다 "담당자를 녹음에서 찾지 못했어요. 직접 입력해 주세요"처럼 사용자가 다음에 할 일을 알 수 있는 문장이 좋습니다.

객관식 퀴즈

Q. 외부 사용자가 1시간짜리 강의 녹음을 업로드하면 요약을 생성하는 웹앱입니다. 처리에는 3~5분이 걸립니다. 처리 방식으로 가장 알맞은 것은 무엇인가요?

• 1. 처리가 끝날 때까지 로딩 화면을 띄우고 동기로 응답한다
• 2. 업로드 직후 접수 응답을 주고 작업 큐에서 처리한 뒤, 처리 상태를 보여 주고 완료되면 메일로 알린다
• 3. 매일 오전 8시에 그동안 업로드된 녹음을 배치로 처리한다
• 4. 처리 시간을 줄이기 위해 녹음의 앞 10분만 요약한다

- •

  1. 처리가 끝날 때까지 로딩 화면을 띄우고 동기로 응답한다

- •

  2. 업로드 직후 접수 응답을 주고 작업 큐에서 처리한 뒤, 처리 상태를 보여 주고 완료되면 메일로 알린다

- •

  3. 매일 오전 8시에 그동안 업로드된 녹음을 배치로 처리한다

- •

  4. 처리 시간을 줄이기 위해 녹음의 앞 10분만 요약한다

Chapter04

코드 없이 검증하기

지금까지 정한 제공 채널, 처리 방식, 출력 형식, 오류 대응은 모두 가정입니다. 사용자가 정말 그 도구 안에서 쓰는지, 출력 형식이 다음 작업에 맞는지, 어디서 막히는지는 사용성 테스트로 확인합니다. 코드를 작성한 뒤에 확인하면 채널이 틀렸다는 것을 알았을 때 이미 며칠을 쓴 뒤이므로, 코드 없이 테스트하는 방법을 씁니다.

오즈의 마법사 테스트(Wizard of Oz prototyping)는 시스템이 하는 일을 사람이 뒤에서 대신 수행하는 방법입니다. 메신저 봇이라면 채팅방에서 작성자가 봇 계정인 것처럼 응답하고, 웹앱이라면 종이나 피그마로 그린 화면을 순서대로 보여 줍니다. LLM이 없어도 채널과 흐름을 검증할 수 있습니다.

기성 AI 도구 프로토타입은 시스템 프롬프트, 출력 형식, 섹션 3에서 정한 오류 대응 규칙을 AI 도구의 프로젝트에 설정해 사용자에게 써 보게 하는 방법입니다. 실제 LLM 응답 품질까지 함께 검증할 수 있고, 섹션 2에서 고른 첫 채널이 기성 AI 도구라면 이 프로토타입이 그대로 첫 버전이 됩니다.

코딩 에이전트에게 맡길 때: 프로토타입용 시스템 프롬프트

```
<기능을 한 문장으로>를 기성 AI 도구의 프로젝트로 먼저 테스트하려고 해. 프로젝트에 넣을 시스템 프롬프트를 써 줘.
- 대상 사용자: <사용 맥락에서 정한 사용자>
- 출력 형식: <항목, 순서, 길이, 값이 없을 때의 표기>
- 오류 대응 규칙: <정한 장치. 예: 결정 사항마다 근거 문장을 붙인다, 담당자가 없으면 추측하지 말고 '찾지 못함'으로 쓴다>
처음 사용하는 사람에게 무엇을 입력하면 되는지 안내하는 첫 메시지도 함께 써 줘.
```

직접 확인할 곳: 오즈의 마법사 테스트 진행

1. 짝과 역할을 나눕니다. 한 사람은 테스트 참여자, 한 사람은 진행자입니다. 15분씩 바꿔 두 번 합니다.
2. 진행자는 상황 하나만 알려 줍니다. 예: "방금 오전 강의가 끝났고 점심을 먹으러 가는 길입니다. 오늘 배운 내용을 복습해 두고 싶어요." 기능 사용법은 설명하지 않습니다.
3. 참여자는 정한 채널(채팅방, 종이 화면, 기성 AI 도구)로 실제처럼 요청합니다. 입력할 자료가 없으면 짧게 만들어 씁니다.
4. 진행자는 정한 출력 형식대로 직접 응답을 작성해 돌려주고, 테스트 노트를 채웁니다. 테스트 중에는 참여자를 돕지 않습니다.

테스트 노트

관찰 항목	기록
설명 없이 처음 시도한 행동	
멈추거나 질문한 지점	
결과를 받은 직후의 행동 (공유, 수정, 무시)	
결과에서 수정한 부분	
참여자가 말한 요청 사항	

노트는 참여자가 말한 것보다 한 행동을 먼저 봅니다. "좋네요"라고 하면서 결과의 절반을 지우고 다시 쓰는 참여자도 있습니다. 결과를 그대로 공유했다면 출력 형식이 맞은 것이고, 수정한 뒤 공유했다면 수정한 부분이 출력 형식이나 오류 대응으로 돌아갈 항목입니다. 처음에 무엇을 할지 몰라 멈췄다면 제공 채널이나 첫 안내 문구를 바꿀 지점입니다. 두 참여자에게서 같은 문제가 나오면 바꾸고, 한 사람에게서만 나왔다면 기록해 두었다가 파일럿에서 다시 확인합니다.

진행 예시: 오즈의 마법사 테스트

아래는 강의 복습 노트 기능을 "스터디 단체 대화방에 있는 봇"으로 제공한다고 가정하고 오즈의 마법사 테스트를 진행한 예시입니다. 진행 과정을 보여 주려고 만든 예시이며, 실제 테스트에서는 참여자의 반응이 다르게 나옵니다. 진행자는 메신저에서 참여자와 새 대화방을 만들고 대화방 이름을 "복습봇"으로 바꾼 뒤, 봇인 것처럼 응답합니다. 응답은 섹션 3에서 정한 출력 형식대로 진행자가 직접 씁니다.

> 진행자: 방금 오전 강의가 끝났고 점심을 먹으러 가는 길이에요. 오늘 배운 내용을 복습해 두고 싶어요. (상황 안내는 여기까지)
>
> 참여자: (노트 앱의 필기를 복사해 붙여 넣는다) 이거 정리해 줘
>
> 복습봇: 오늘 필기에서 핵심 개념 4개를 찾았어요. ① HTTP 요청: 클라이언트가 서버에 보내는 메시지 (필기 3번째 줄) ② 상태 코드: … / 퀴즈 1. 서버가 요청을 처리하지 못했을 때 돌려주는 상태 코드 범위는? ① 2xx ② 3xx ③ 4xx ④ 5xx · 정답: ④
>
> 참여자: 아, 정답이 바로 보여서 풀기 전에 봐 버렸어요.
>
> 참여자: 이거 내일 아침에 다시 보려면 어디서 찾아요? 단톡방이면 금방 묻힐 것 같은데.

15분 테스트가 끝난 뒤 진행자가 채운 테스트 노트입니다.

관찰 항목	기록
설명 없이 처음 시도한 행동	사진을 찍지 않고 노트 앱 필기를 복사해 붙여 넣었다
멈추거나 질문한 지점	다음 날 결과를 어디서 다시 찾는지 물었다
결과를 받은 직후의 행동	핵심 개념은 훑어보고 넘겼고, 퀴즈를 풀려다 정답을 먼저 봤다
결과에서 수정한 부분	없음
참여자가 말한 요청 사항	퀴즈만 따로 다시 받고 싶다

이 노트로 바꾼 것은 세 가지입니다. 출력 형식에서 정답을 분리해 "정답"이라고 보낼 때만 보내기로 했고, 첫 안내 문구에서 사진 올리기보다 붙여 넣기를 앞에 두었습니다. 결과를 다음 날 다시 찾기 어렵다는 관찰은 채널 문제입니다. 대화가 계속 쌓이는 단체 대화방은 결과를 보관하는 데 맞지 않으므로, 대화 기록이 기능별로 남는 기성 AI 도구 프로젝트를 첫 채널로 바꿨습니다.

진행 예시: 기성 AI 도구 프로토타입

오즈의 마법사 테스트에서 바꾼 채널로 기성 AI 도구 프로토타입을 만든 예시입니다. AI 도구에서 새 프로젝트를 만들고, 프로젝트 지침(시스템 프롬프트) 칸에 아래 내용을 붙여 넣습니다. 메뉴 이름은 도구마다 다르지만, 대부분 프로젝트 설정 화면에 지침이나 지시 사항 칸이 있습니다.

프로젝트 지침 예시

```
너는 강의 필기로 복습 자료를 만드는 도우미다.
사용자는 강의를 듣는 수강생이고, 강의가 끝난 직후 필기를 붙여 넣거나 사진으로 올린다.

[출력 형식]
1. 핵심 개념: 필기에 나온 개념 3~5개. 개념마다 한 줄 정의와 근거(필기 몇 번째 줄인지, 또는 원문 인용)
2. 헷갈리기 쉬운 점: 1~2개
3. 퀴즈: 객관식 3문항, 서술형 1문항. 정답과 해설은 쓰지 않는다
사용자가 "정답"이라고 보내면 그때 정답과 해설을 보낸다. 해설에도 필기 근거를 붙인다.

[규칙]
- 필기에 없는 내용은 개념, 퀴즈로 만들지 않는다.
- 필기만으로 정의를 쓸 수 없으면 "필기에 정의가 없어요. 강의 자료를 확인해 주세요"라고 쓴다.
- 개념을 3개 찾지 못하면 찾은 개수만 쓰고 그 사실을 알린다.
- 사진에서 읽을 수 없는 부분은 추측하지 않고 "읽지 못한 부분"으로 표시한다.

[첫 메시지]
강의가 끝나면 필기를 붙여 넣거나 사진으로 올려 주세요. 핵심 개념과 퀴즈를 만들어 드릴게요. 정답은 "정답"이라고 보내면 알려 드려요.
```

참여자에게는 프로젝트를 공유하거나, 진행자의 노트북에서 프로젝트를 연 채로 넘겨줍니다. 이번에는 진행자가 응답을 쓰지 않으므로, 참여자가 실제 오늘 오전 강의의 필기를 넣게 하면 실제 데이터로 응답 품질까지 볼 수 있습니다. 다른 사람의 이름이나 연락처가 들어간 필기는 넣지 않습니다. 아래는 15분 테스트로 나온 관찰과 그에 따라 바꾼 지침의 예시입니다.

관찰	원인	바꾼 지침
개념 정의 하나에 필기에 없는 설명이 붙었고, 출처가 "일반 지식"으로 적혔다	근거가 없을 때의 규칙이 정의에만 있고 해설에는 없었다	"해설과 보기도 필기에 나온 내용으로만 만든다"를 추가
객관식 오답 보기가 너무 뻔해서 바로 정답을 골랐다	오답 보기를 만드는 규칙이 없었다	"오답 보기는 필기에 나온 다른 개념으로 만든다"를 추가
다음 날 같은 퀴즈를 다시 풀고 싶다고 했다	기성 AI 도구는 틀린 문제를 따로 저장하지 않는다	지침으로 해결할 수 없음. 웹앱 전환 조건 후보로 PRD에 기록

세 번째 관찰처럼 지침을 고쳐도 해결되지 않는 요구사항은 그 채널의 한계입니다. 이런 요구사항이 파일럿에서도 반복해서 나오면, 섹션 2에서 적은 전환 조건에 해당하는지 보고 다음 채널로 옮길지 정합니다.

주관식 퀴즈

Q. [이해 점검] 테스트 노트에서 가장 예상과 달랐던 관찰은 무엇인가요? 그 관찰 때문에 바꾸기로 한 것이 제공 채널, 처리 방식, 출력 형식, 오류 대

응 가운데 어디에 해당하는지 적어 보세요.

Chapter05

아키텍처를 정하지 않으면

섹션 2의 제공 채널 가운데 웹앱을 고르면 클라이언트, 백엔드, 데이터 저장, LLM 호출을 모두 직접 설계해야 합니다. 기성 AI 도구에서는 도구가 대신 정해 주던 것들입니다. 코딩 에이전트에게 웹앱 개발을 요청하면 이 설계도 에이전트가 정하는데, 요청에 없는 부분은 가장 빨리 동작하는 방법으로 채우기 쉽습니다.

요청에 없던 항목	에이전트가 고르기 쉬운 방법	나중에 드러나는 문제
API 키 위치	클라이언트 코드에 하드코딩	브라우저 개발자 도구로 키가 노출되어 타인이 내 비용으로 호출한다
데이터 저장소	서버 메모리나 브라우저 로컬 스토리지	서버 재시작이나 기기 변경 시 데이터가 사라진다
데이터 소유자	정하지 않음	다른 사용자의 데이터를 조회하거나 수정할 수 있다
외부 API 실패 처리	처리하지 않음	LLM 응답이 늦거나 실패하면 화면이 멈추거나 빈 데이터가 저장된다
모듈 분리	한 파일에 모두 작성	기능 하나를 수정하다 관계없는 기능이 함께 깨진다

어느 것도 처음 화면이 뜰 때는 드러나지 않고, 사용자가 늘거나 재배포하거나 기능을 수정할 때 드러납니다. 그래서 웹앱은 코드를 쓰기 전에 아키텍처를 정해 요청에 넣거나, 이미 만든 앱이라면 지금의 아키텍처를 문서화해 문제를 찾아야 합니다. 아키텍처는 보통 아키텍처 다이어그램, 시퀀스 다이어그램, 데이터 모델 세 가지로 나눠 문서화합니다. 오후의 예시는 사진을 올리면 AI가 판매글 초안을 생성하는 중고거래 앱입니다. 외부 사용자 대상이라 오전의 다섯 채널 가운데 웹앱(여기서는 휴대폰 앱)을 고른 경우입니다.

아키텍처 다이어그램

아키텍처 다이어그램은 시스템을 이루는 구성 요소와 그 사이의 호출 관계를 그린 그림입니다. 중고거래 앱이라면 클라이언트(휴대폰 앱)가 백엔드 API를 호출하고, 백엔드가 다시 LLM API, 오브젝트 스토리지, 데이터베이스, 작업 큐를 호출합니다.



그림: 중고거래 앱의 아키텍처 다이어그램. 클라이언트, 백엔드 모듈 다섯 개, 외부 의존성 네 개

백엔드 안은 모듈로 나눴습니다. 판매글 API, AI 초안 생성, 시세 조회처럼 책임 하나에 모듈 하나를 두면 시세 조회 로직을 수정할 때 `price` 모듈만 보면 됩니다. 모듈 이름 아래의 `listings`, `ai_draft` 는 코드에서 그 모듈을 찾을 때 쓰는 파일이나 디렉터리 이름입니다. 작업 큐와 워커만 점선인 것은 사용자의 요청과 분리되어 비동기로 실행되기 때문입니다.

가장 먼저 확인하는 것은 시크릿의 위치입니다. LLM API 키와 스토리지 접근 키는 백엔드에만 있어야 합니다. 클라이언트 코드는 사용자의 기기로 내려가므로 그 안에 넣은 키는 누구나 추출할 수 있습니다. 바이브코딩으로 만든 앱을 문서화해 보면 키가 클라이언트 쪽에 있는 경우가 꽤 많습니다.

시퀀스 다이어그램

시퀀스 다이어그램은 사용자 동작 하나가 처리되는 동안 구성 요소 사이에 오가는 호출을 시간 순서대로 그린 그림입니다. 맨 위에 구성 요소를 가로로 놓고, 아래로 내려가며 요청(실선)과 응답(점선)을 화살표로 적습니다. 조건에 따라 흐름이 갈리는 부분은 `alt` 상자로 표시합니다.

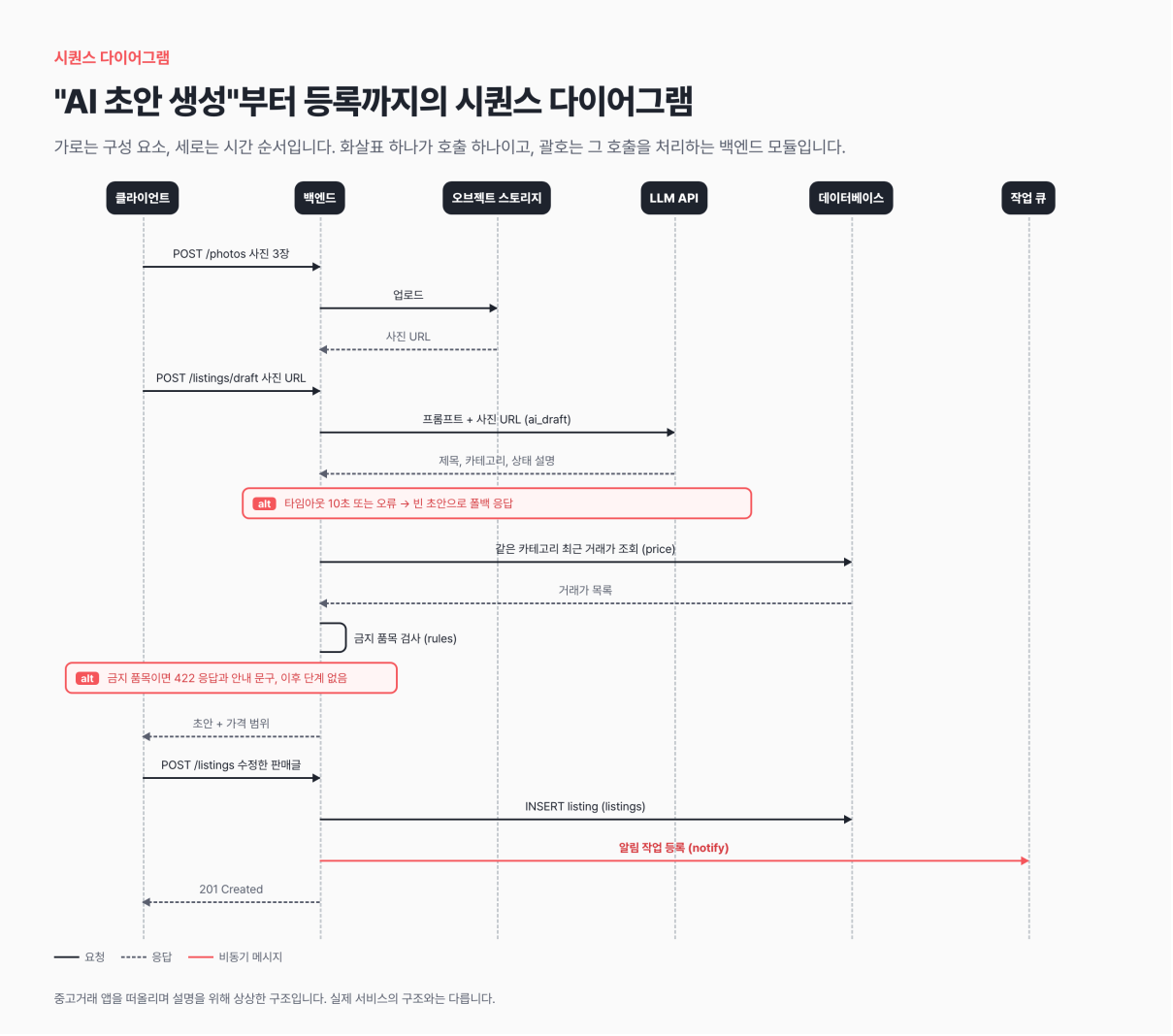


그림: "AI 초안 생성"부터 판매글 등록까지의 시퀀스 다이어그램

시퀀스 다이어그램을 그리면 아키텍처 다이어그램만으로는 보이지 않던 것이 보입니다. LLM API 호출은 몇 초가 걸리고 호출마다 비용이 발생하므로, 사용자가 버튼을 여러 번 누르면 비용도 그만큼 늘어납니다. 그래서 첫 번째 `alt` 상자처럼 타임아웃과 실패 시의 폴백 응답을 정해 두고, 클라이언트에서는 요청 중에 버튼을 비활성화합니다. 금지 품목에 걸리면 두 번째 `alt` 상자처럼 422 응답과 안내 문구를 돌려주고 이후 호출은 일어나지 않습니다. 근처 사용자 알림은 수백 명에게 보내야 할 수 있어서, 등록 요청 안에서 처리하면 판매자가 응답을 오래 기다리게 됩니다. 그래서 작업 큐에 등록만 하고 바로 `201 Created` 를 돌려줍니다.

데이터 모델

데이터 모델은 시스템이 저장하는 엔티티와 그 속성을 정리한 문서입니다. 여기서는 엔티티마다 생성 시점, 소유자, 저장소, 보존 기간만 적은 표로 정리합니다. 화면에 보이는 것과 저

장되는 것은 다릅니다. AI 초안은 화면에 보이지만 저장하지 않기로 했고, 알림 작업은 화면에 보이지 않지만 처리될 때까지 작업 큐에 저장됩니다.



그림: 중고거래 앱의 데이터 모델. 엔티티별 생성 시점, 소유자, 저장소, 보존 기간

소유자 칸은 인가(authorization)로 이어집니다. 판매글의 소유자가 판매자라면, 판매글 수정이나 삭제 요청이 올 때마다 백엔드는 요청한 사용자가 그 판매글의 소유자인지 검사해야 합니다. 보존 기간 칸은 개인정보 처리와 스토리지 비용으로 이어집니다. 보존 기간을 정하지 않으면 탈퇴한 사용자의 사진이 스토리지에 계속 남고 저장 비용도 계속 발생합니다. 거래 내역처럼 소유자가 둘인 엔티티는 한 사람의 삭제 요청만으로 바로 지우면 안 되는 경우도 있어서, 표를 채우다 보면 이런 판단이 필요한 곳이 드러납니다.

객관식 퀴즈

Q. 중고거래 앱의 시퀀스 다이어그램에서, 근처 사용자 알림을 등록 요청 안에서 보내지 않고 작업 큐에 등록한 이유로 가장 알맞은 것은 무엇인가요?

- 1. 알림 데이터는 저장할 필요가 없어서
- 2. 알림 대상이 많으면 등록 요청의 응답 시간이 그만큼 길어지므로, 등록은 바로 응답하고 알림은 워커가 비동기로 처리하게 하려고
- 3. 알림 문구를 LLM이 생성해야 해서
- 4. 알림 기능을 첫 버전에서 제외하려고**제출**

- •

1. 알림 데이터는 저장할 필요가 없어서

- •

  2. 알림 대상이 많으면 등록 요청의 응답 시간이 그만큼 길어지므로, 등록은 바로 응답하고 알림은 워커가 비동기로 처리하게 하려고

- •

  3. 알림 문구를 LLM이 생성해야 해서

- •

  4. 알림 기능을 첫 버전에서 제외하려고

주관식 퀴즈

Q. [이해 점검] 중고거래 앱에서 LLM API가 한 시간 동안 장애를 겪었다고 해 봅시다. 시퀀스 다이어그램과 데이터 모델을 근거로, 사용자에게 영향을 주는 기능과 영향을 주지 않는 기능을 나눠 적어 보세요.

Chapter06

시퀀스 다이어그램부터

섹션 5에서 읽은 세 문서를 이번에는 오전에 정한 내 서비스로 직접 설계합니다. 아직 만들지 않은 서비스이므로 코드는 필요 없습니다. 섹션 2에서 첫 채널로 기성 AI 도구나 메신저 봇을 골랐다면, 전환 조건이 충족되어 웹앱으로 옮긴다고 가정하고 설계합니다. 이 결과가 섹션 7 PRD의 아키텍처 항목이 됩니다.

세 문서 가운데 시퀀스 다이어그램을 가장 먼저 그립니다. 사용자 동작 하나에서 출발하므로 오전에 정한 사용 맥락과 처리 방식을 그대로 이어서 그릴 수 있고, 그리고 나면 필요한 구성 요소와 저장할 데이터가 함께 드러나기 때문입니다. 순서는 아래와 같습니다.

직접 확인할 곳: 설계 순서

1. 시퀀스 다이어그램: 내 서비스에서 가장 중요한 사용자 동작 하나를 고릅니다. 강의 복습 노트라면 "필기를 붙여 넣고 퀴즈 만들기"입니다. 빈 양식 맨 위에 그 동작에 관련된 구성 요소(클라이언트, 백엔드, LLM API, 데이터베이스 등)를 적고, 요청과 응답을 시간 순서대로 화살표로 그립니다. 섹션 3에서 정한 처리 방식과 오류 대응을 `alt` 상자로 넣습니다.
2. 아키텍처 다이어그램: 시퀀스 다이어그램에 나온 구성 요소를 모아 옮겨 그립니다. 백엔드는 화살표마다 하는 일을 보고 모듈로 나누고, 시크릿이 놓일 위치를 표시합니다.
3. 데이터 모델: 시퀀스 다이어그램에서 데이터베이스나 스토리지로 가는 화살표를 찾아, 그 화살표가 저장하는 것을 엔티티로 적습니다. 엔티티마다 소유자와 보존 기간을 정합

니다. 화면에만 보이고 저장하지 않기로 한 것도 "저장 안 함"으로 적습니다.

4. 점검: 섹션 5의 첫 번째 표에 있는 다섯 항목(API 키 위치, 데이터 저장소, 데이터 소유자, 외부 API 실패 처리, 모듈 분리)을 내 설계에서 모두 정했는지 확인합니다. 정하지 않은 항목은 코딩 에이전트가 대신 정하게 되는 항목입니다.



그림: 시퀀스 다이어그램 빈 양식

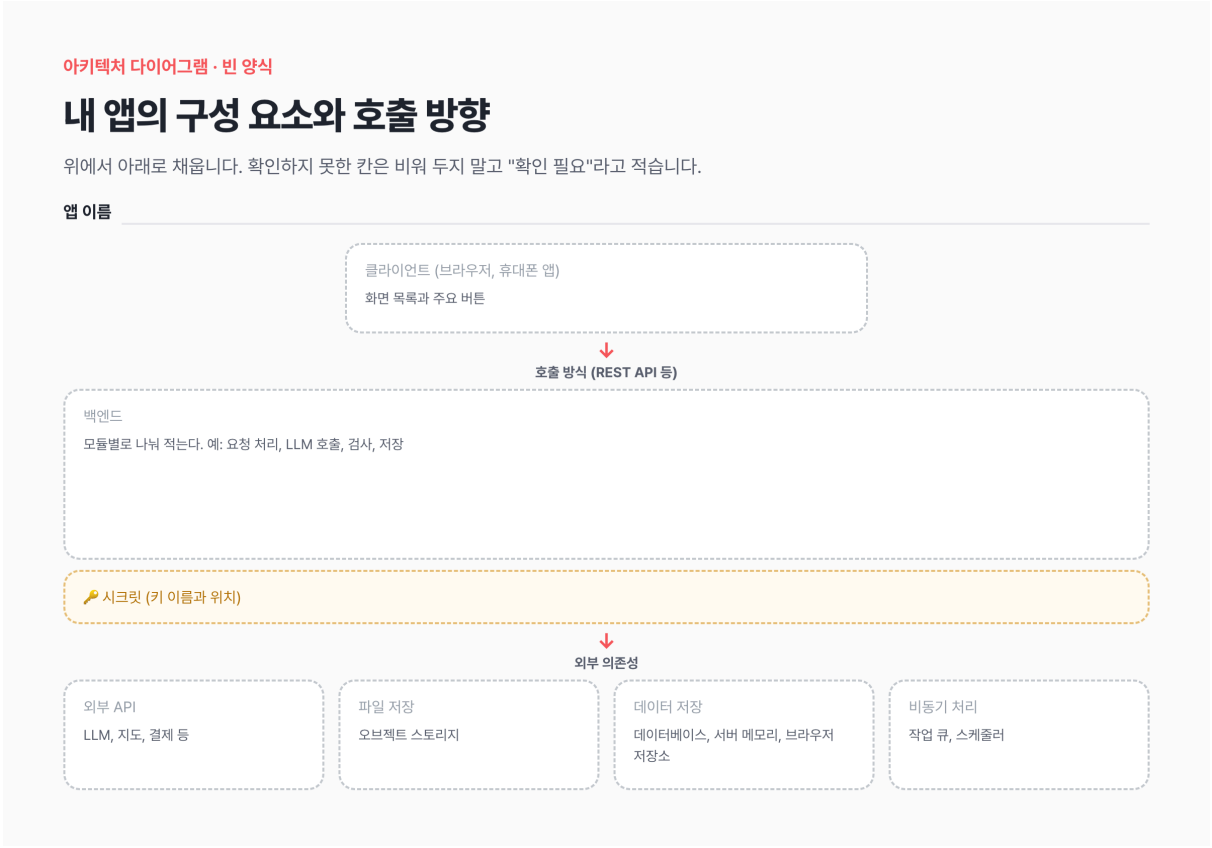


그림: 아키텍처 다이어그램 빈 양식

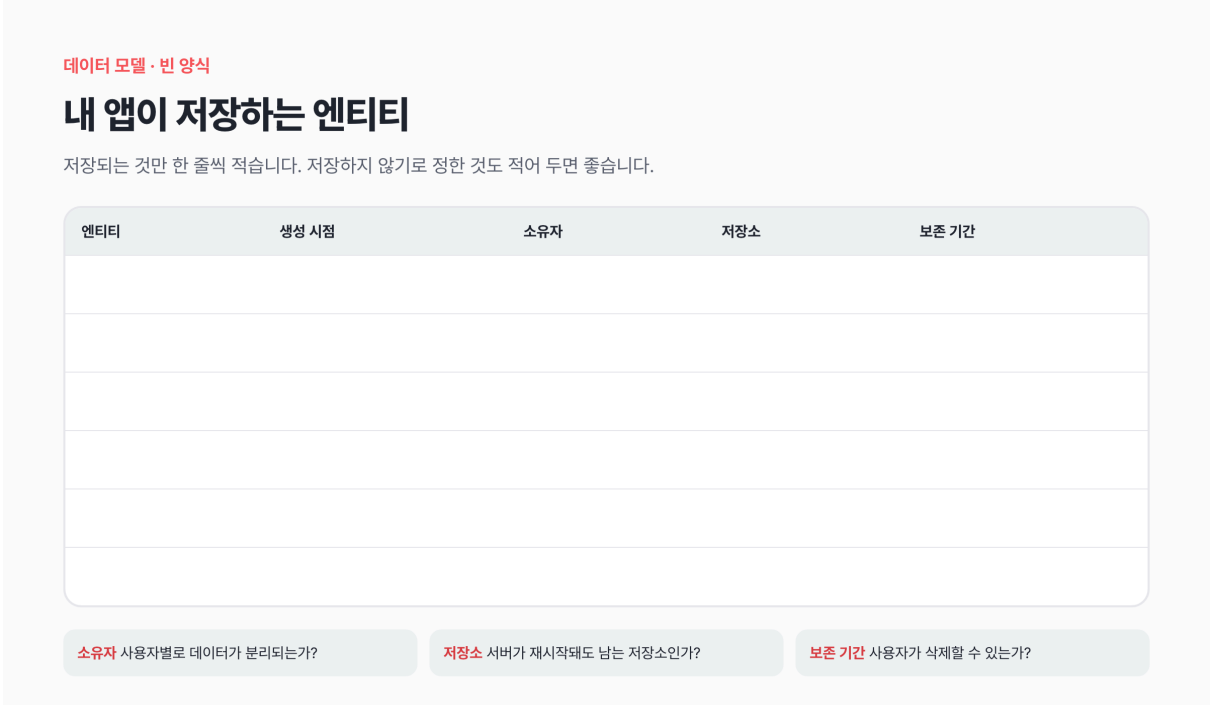


그림: 데이터 모델 빈 양식

설계 리뷰 받기

다 그렸으면 코딩 에이전트에게 설계 리뷰를 요청합니다. 이때 에이전트가 설계를 대신 바꾸지 않고, 빠진 항목을 질문으로만 알려 주게 합니다. 결정은 내가 하고, 에이전트는 놓친 곳을 찾는 데 씁니다.

코딩 에이전트에게 맡길 때: 설계 리뷰

```
아래는 내가 설계한 서비스의 시퀀스 다이어그램, 아키텍처 다이어그램, 데이터 모델이야. 설계를 고치거나
대신 정하지 말고,
빠졌거나 정하지 않은 항목을 질문 형태로 다섯 개 이내로 알려 줘. 특히 API 키 위치, 데이터 저장소와
소유자, 외부 API가 실패했을 때의 흐름을 봐 줘.

<세 문서 내용을 붙여 넣는다. 사진으로 찍었다면 이미지를 첨부한다>
```

받은 질문 가운데 동의하는 것은 설계에 반영하고, 동의하지 않는 것은 그 이유를 한 줄로 적어 둡니다. 에이전트의 질문이 항상 맞는 것은 아니고, 오전에 정한 사용 맥락이나 파일럿 범위를 모르는 상태에서 나온 질문일 수 있습니다.

주관식 퀴즈

Q. [이해 점검] 시퀀스 다이어그램을 그리고 나서 아키텍처 다이어그램과 데이터 모델을 채울 때, 시퀀스에서 처음 드러난 구성 요소나 데이터가 있었나요? 있었다면 무엇이었고, 그것을 정하지 않은 채 코딩 에이전트에게 개발을 요청했다면 어떻게 됐을지 섹션 5의 첫 번째 표를 근거로 적어 보세요.

Chapter07

/강의정리 챕터 06. 시퀀스 다이어그램부터

1. 강의의 목적

이번 강의의 핵심은 내가 만들 AI 서비스를 직접 설계하고, 코딩 에이전트에게 개발을 맡기기 전에 설계의 빈틈을 확인하는 것입니다.

챕터 05에서는 웹앱을 설계할 때 필요한 세 가지 문서를 배웠습니다.

1. 시퀀스 다이어그램: 기능이 어떤 순서로 처리되는가?
2. 아키텍처 다이어그램: 어떤 구성 요소가 필요한가?
3. 데이터 모델: 어떤 데이터를 저장하고 누가 소유하는가?

이번에는 이 문서들을 실제로 만들 서비스에 적용합니다.

중요한 차이는 이번에는 아키텍처부터 그리지 않고, 시퀀스 다이어그램부터 그린다는 점입니다. 사용자의 행동 하나에서 시작하면 실제로 필요한 구성 요소와 데이터가 자연스럽게 드러나기 때문입니다.

2. 중학생도 이해할 수 있는 설명

AI 서비스를 음식점에 비유해 보겠습니다.

처음부터 식당의 모든 공간과 장비를 설계하면 정작 손님이 어떤 순서로 주문하고 음식을 받는지 놓칠 수 있습니다.

따라서 먼저 손님 한 명의 주문 과정을 그려 봅니다.

1

시퀀스 다이어그램부터 작성

손님이 주문하고, 주방이 조리하고, 직원이 음식을 전달하는 순서를 그립니다.

2

아키텍처 다이어그램 작성

주문대, 주방, 창고, 직원 등 실제로 필요한 구성 요소를 정리합니다.

3

데이터 모델 작성

주문서와 재고를 어디에 보관하고 누가 관리할지 정합니다.

AI 서비스도 마찬가지입니다. 사용자가 버튼을 누른 뒤 어떤 일이 일어나는지부터 그리면, 필요한 서버와 저장소, 외부 AI 호출, 오류 대응을 구체적으로 정할 수 있습니다.

3. 직접 설계하는 순서

1단계. 시퀀스 다이어그램

가장 중요한 사용자 동작을 하나 선택합니다.

예를 들어 강의 복습 노트 서비스라면 다음과 같습니다.

> 사용자가 강의 필기를 붙여 넣고 퀴즈 만들기를 요청한다.

이 동작을 중심으로 구성 요소를 나열하고 요청과 응답을 시간순으로 연결합니다.

강의 복습 노트 서비스의 기본 흐름

1. 사용자가 필기를 입력하고 퀴즈 생성을 요청합니다.
2. 클라이언트가 백엔드에 요청을 전달합니다.
3. 백엔드가 입력을 검증하고 LLM API를 호출합니다.
4. LLM이 핵심 개념과 퀴즈를 생성합니다.
5. 백엔드가 결과를 검증한 뒤 클라이언트에 반환합니다.
6. 사용자는 결과를 확인하고, 설계에 따라 필요한 데이터만 저장합니다.

개념을 이해하기 위한 예시입니다. 실제 서비스의 저장 방식과 구성 요소는 직접 결정해야 합니다.

여기에 앞선 강의에서 배운 처리 방식과 오류 대응을 추가합니다.

- LLM 응답이 늦으면 어떻게 할까요?
- LLM 호출이 실패하면 어떤 안내를 보여 줄까요?
- 필기가 비어 있거나 읽을 수 없다면 어떻게 할까요?
- 사용자가 버튼을 여러 번 누르면 어떻게 할까요?
- 결과를 저장하지 않기로 했다면 어디까지 보관할까요?

조건에 따라 흐름이 달라지는 부분은 시퀀스 다이어그램의 `alt` 영역으로 표현할 수 있습니다. `alt`는 쉽게 말해 **"이 조건이면 A, 다른 조건이면 B"**라는 분기입니다.

2단계. 아키텍처 다이어그램

시퀀스 다이어그램에 등장한 구성 요소를 모아 시스템 전체 구조를 그립니다.

예를 들어 다음 구성 요소가 필요할 수 있습니다.

- 클라이언트: 사용자가 필기를 입력하고 결과를 보는 화면
- 백엔드: 입력 검증, AI 호출, 결과 검증
- LLM API: 복습 자료와 퀴즈 생성
- 데이터베이스: 사용자 정보나 학습 이력을 저장하는 경우
- 작업 큐와 워커: 시간이 오래 걸리는 작업을 별도로 처리하는 경우

여기서 모든 서비스에 데이터베이스나 작업 큐가 반드시 필요한 것은 아닙니다. 실제 요구사항에 필요한 구성 요소만 선택하는 것이 중요합니다.

백엔드도 책임에 따라 모듈로 분리합니다. 예를 들어 `quiz`는 퀴즈 생성 관련 처리, `study_notes`는 복습 노트 관련 처리를 담당하도록 나눌 수 있습니다.

그리고 LLM API 키 등 비밀정보는 백엔드에서 관리하도록 명시합니다.

3단계. 데이터 모델

이번에는 시퀀스 다이어그램에서 데이터베이스나 파일 저장소로 향하는 화살표를 찾아봅니다.

각 화살표가 저장하는 데이터를 데이터 모델에 기록합니다.

데이터	생성 시점	소유자	저장 위치	보존 기간
강의 필기 원문	사용자가 입력할 때	사용자	저장 안 함 또는 별도 저장소	정책에 따라 결정
AI 복습 결과	생성 완료 시	사용자 작업에 연결	저장 안 함 또는 데이터베이스	정책에 따라 결정
퀴즈 풀이 기록	사용자가 답을 제출할 때	해당 사용자	데이터베이스	학습 이력 정책에 따라 결정
오류 로그	처리 실패 시	서비스 운영 주체	로그 저장소	운영·보안 정책에 따라 결정

위 표는 예시이며, 모든 항목을 반드시 저장해야 한다는 의미는 아닙니다.

특히 화면에 표시된다고 해서 반드시 저장해야 하는 것은 아닙니다. 반대로 화면에 보이지 않더라도 오류 로그나 비동기 작업처럼 내부적으로 저장해야 하는 데이터가 있을 수 있습니다.

데이터마다 소유자와 보존 기간을 정해야 사용자 권한, 개인정보 보호, 저장 비용 문제도 함께 검토할 수 있습니다.

4. 설계 점검: 다섯 가지 항목

세 문서를 작성한 뒤에는 다음 항목이 모두 정해졌는지 확인합니다.

아키텍처 점검표

0/5 확인

- [ ] API 키 위치
  LLM API 키와 스토리지 접근 키가 클라이언트에 노출되지 않도록 했나요?
- [ ] 데이터 저장소
  필기, 결과, 학습 기록 등을 어디에 저장할지 또는 저장하지 않을지 정했나요?

- [ ] 데이터 소유자
  사용자별 데이터의 소유자와 조회·수정·삭제 권한을 정했나요?
- [ ] 외부 API 실패 처리
  타임아웃, 오류 안내, 재시도 또는 대체 흐름을 정했나요?
- [ ] 모듈 분리
  기능별 책임을 나눠 수정과 유지보수가 쉽도록 했나요?

정하지 않은 항목을 그대로 두면 코딩 에이전트가 대신 결정하게 됩니다. 그 결정이 처음에는 잘 작동하더라도 사용자가 늘거나 기능을 수정할 때 문제가 드러날 수 있습니다.

5. 코딩 에이전트에게 설계 리뷰 받기

설계를 완성했다면 에이전트에게 검토를 요청할 수 있습니다. 여기서 중요한 점은 에이전트가 설계를 임의로 바꾸게 하지 않는 것입니다.

AI 서비스 설계 리뷰 요청

아래는 제가 설계한 서비스의 시퀀스 다이어그램, 아키텍처 다이어그램, 데이터 모델입니다.

설계를 직접 수정하거나 대신 결정하지 마세요.

빠졌거나 아직 결정되지 않은 사항을 질문 형태로 최대 5개까지 알려 주세요.

특히 다음 항목을 검토해 주세요.

1. API 키와 비밀정보의 위치
2. 데이터 저장소와 데이터 소유자
3. 외부 API 실패 시 처리 흐름
4. 구성 요소 사이의 호출 관계
5. 모듈 분리와 데이터 보존 정책

질문마다 왜 확인이 필요한지 한 문장으로 설명해 주세요.

제가 검토한 뒤 직접 설계를 수정하겠습니다.

[여기에 세 문서의 내용 또는 이미지 첨부]

에이전트가 질문했다고 해서 모두 반영할 필요는 없습니다. 질문을 검토한 뒤 동의하는 내용은 설계에 반영하고, 동의하지 않는 내용은 이유를 기록합니다.

예를 들어 “사용자별 학습 이력을 저장해야 하지 않나요?”라는 질문을 받았다면, 학습 이력이 첫 버전의 범위에 포함되는지 먼저 판단해야 합니다. MVP에서 저장 기능을 제외하기로 했다면, 그 이유와 향후 추가 조건을 기록하면 됩니다.

6. AI 개발자와 1인 창업가에게 왜 중요할까요?

이번 강의는 단순히 다이어그램을 그리는 기술을 배우는 시간이 아닙니다. AI에게 개발을 맡기더라도 서비스의 중요한 결정을 직접 내리는 연습입니다.

특히 1인 AI 사업에서는 다음과 같은 이점이 있습니다.

- 개발을 시작하기 전에 필요한 구성 요소를 파악할 수 있습니다.
- 기능을 추가할 때 기존 시스템에 미칠 영향을 예상할 수 있습니다.
- 코딩 에이전트가 선택한 구조를 검토하고 질문할 수 있습니다.
- 고객 데이터와 API 비용을 관리하는 기준을 만들 수 있습니다.
- 이후 PRD에 들어갈 아키텍처 요구사항을 구체화할 수 있습니다.

ASP나 My-Story-Maker 같은 서비스를 개발할 때도 동일하게 적용할 수 있습니다. 처음에는 간단한 AI 기능으로 시작하더라도 사용자의 자료를 저장하거나 여러 사람이 함께 사용하는 단계로 넘어가면 데이터 소유권과 접근 권한이 중요해지기 때문입니다.

7. 이해 점검 퀴즈

Q1. 시퀀스 다이어그램을 먼저 작성하는 이유로 가장 적절한 것은 무엇인가요?

세 문서 중 가장 예쁘게 그릴 수 있기 때문입니다.

사용자 동작의 처리 순서를 먼저 확인하면 필요한 구성 요소와 데이터가 드러나기 때문입니다.

시퀀스 다이어그램만 있으면 나머지 문서가 필요 없기 때문입니다.

코딩 에이전트가 설계 결정을 모두 대신할 수 있기 때문입니다.

정답 확인

8. 주관식 퀴즈 예시 답안

질문: 시퀀스 다이어그램을 그리고 나서 아키텍처 다이어그램과 데이터 모델을 채울 때, 시퀀스에서 처음 드러난 구성 요소나 데이터가 있었나요? 그것을 정하지 않은 채 코딩 에이전트에게 개발을 요청했다면 어떻게 됐을까요?

강의의 복습 노트 서비스를 예로 들면 다음과 같이 답할 수 있습니다.

> 시퀀스 다이어그램을 그리면서 LLM API뿐 아니라 오류 로그와 학습 이력의 저장 여부, 외부 API가 실패했을 때의 대체 흐름도 검토할 필요가 있음을 발견했습니다. 이런 사항을 정하지 않은 채 코딩 에이전트에게 개발을 요청했다면, 에이전트가 임시 저장소를 선택하거나 오류 처리를 생략할 수 있습니다. 또한 사용자별 데이터 소유권을 정하지 않으면 다른 사용자가 데이터에 접근하는 문제도 발생할 수 있습니다. 따라서 API 키 위치, 저장소, 데이터 소유자, 외부 API 실패 처리, 모듈 분리를 미리 결정하고 설계 리뷰를 통해 누락된 항목을 확인해야 합니다.

실제 답변에서는 본인이 설계하면서 새롭게 발견한 구성 요소나 데이터를 구체적으로 적는 것이 좋습니다.

9. 반드시 기억할 내용

1. 시퀀스 다이어그램부터 시작합니다. 중요한 사용자 동작 하나를 선택해 처리 순서를 그립니다.
2. 아키텍처 다이어그램은 처리 순서에서 필요한 구성 요소를 추려 작성합니다.
3. 데이터 모델은 저장하는 데이터뿐 아니라 저장하지 않는 데이터도 명시합니다.
4. API 키, 저장소, 소유자, 실패 처리, 모듈 분리는 반드시 확인합니다.
5. 코딩 에이전트는 설계의 빈틈을 찾는 검토자로 활용합니다. 중요한 결정은 직접 내립니다.
6. 완성한 세 문서는 PRD의 아키텍처 항목으로 연결됩니다.

한 문장 요약

내 서비스의 핵심 사용자 동작을 시퀀스 다이어그램으로 먼저 설계하고, 그 흐름에서 아키텍처와 데이터 모델을 도출한 뒤, 코딩 에이전트에게 빠진 부분을 질문하도록 해야 합니다.

Chapter08

오늘 배운 것

제공 방식. 제공 채널은 기능이 아니라 사용 맥락이 정합니다. 사용자가 이미 쓰는 도구 안에서 끝나는 채널을 찾고, 개발 부담이 작은 기성 AI 도구로 먼저 검증한 뒤 전환 조건이 생기면 옮깁니다. LLM 처리 시간에 맞춰 동기, 비동기, 스케줄 실행 가운데 처리 방식을 고르고, 오

류의 영향도가 클수록 출처 표시, 휴먼 인 더 루프, 에스컬레이션 같은 대응 장치를 더 넣었습니다.

검증. 오즈의 마법사 테스트와 기성 AI 도구 프로토타입으로 코드 없이 사용성 테스트를 했고, 참여자의 말보다 행동을 기준으로 바꿀 항목을 정했습니다.

아키텍처. 웹앱은 아키텍처 다이어그램, 시퀀스 다이어그램, 데이터 모델로 문서화합니다. 오전에 정한 내 서비스를 시퀀스 다이어그램부터 설계했고, 시퀀스에서 드러난 구성 요소와 저장 대상으로 아키텍처 다이어그램과 데이터 모델을 채웠습니다. 코딩 에이전트에게는 설계를 대신 정하게 하지 않고 빠진 항목을 질문으로 받았습니다.

PRD. 접근 제어, 데이터 처리, 비용, 파일럿 계획, 아키텍처를 한 문서에 모았습니다. 이 과정의 뒤에서 배우는 배포와 운영 기술은 이 문서의 요구사항을 구현하는 방법입니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 내 AI 기능의 사용 경로와, 버튼을 눌렀을 때의 내부 호출 순서를 적었습니다. 오늘 쓴 PRD와 섹션 5에서 읽은 시퀀스 다이어그램을 떠올리며, 처음 답에서 가장 많이 바뀐 생각 하나와 그 근거를 적어 보세요.